Data Storytelling

Dr. Patrick Chester

Spring 2026

Week 06

Announcements

- Adjusted syllabus to reflect current progress
- How are the assignments coming along? Due tonight at 11:59 PM!
- How do you generate a pdf report from a qmd file?
  - Specify output format in the YAML header
    - Eg. `format: pdf`
  - Click the "render" button in RStudio
  - Note: If you are using R Markdown, you'll use `output: pdf` instead.

Predicting Data

Why Predict Data?

- One of the most valued skills data analysts is how to predict outcomes.
  - What will the stock market do tomorrow?
  - What will the weather be like next week?
  - Predicting job growth in the following quarter.
- These predictions are valuable for decision-making, planning, and understanding trends.

How can we predict data?

- Typically, we use statistical models, machine learning algorithms, or time series analysis to make predictions based on historical data.
  - Models are some function mapping input data to output data.

$$Y = f(X) + \epsilon$$

- Where,
  - Y is the outcome we want to predict (ex. GDP, stock price, etc.),
  - X is the input data (ex. historical GDP, stock prices, etc.),
  - f is the function that maps X to Y,
  - ϵ is the error term (unexplained variance)

Modeling Approaches

- The model you use depends on the type of data you have and the problem you're trying to solve.
  - **Continuous outcomes**:
    - *Examples:* GDP, temperature, company revenue, etc.
    - *Models:* Linear regression, support vector regression, or transformer models.
  - **Binary outcomes**:
    - *Examples*: Will a war occur between two countries? Will a person vote for a particular candidate? Will a company go bankrupt?
    - *Models*: Logistic regression, decision trees, or random forests.
  - **Multiclass outcomes**:
    - *Examples*: What type of job will a person have? What type of product will a customer buy? What type of disease does a patient have?
    - *Models*: Multinomial logistic regression, transformer models.

Linear Regression

- Linear regression, or Ordinary Least Squares (OLS), is a simple and widely used method for predicting a continuous outcome variable based on one or more predictor variables.
- Goal is to fit a line to observed data. Does anyone remember the formula for a line?

$$Y = mX + b$$

- In linear regression, we can rewrite this as:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

A fitted Linear Model

- Let's say that we wanted to model the relationship between study time (X) and test scores (Y). We could fit a linear regression model to our data, which would give us estimates for β_0 and β_1.

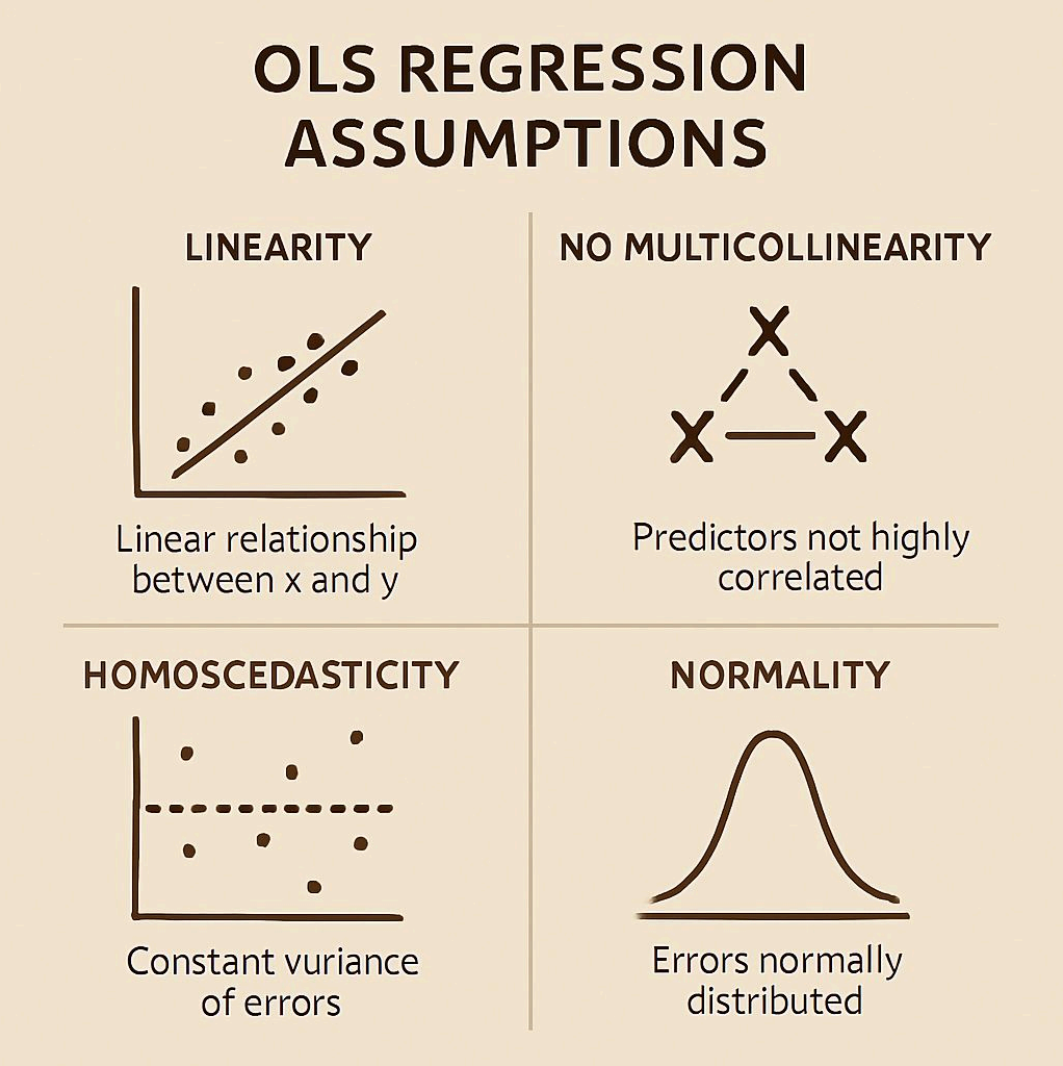
$$\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 X$$

- How would we interpret $\hat{\beta}_0$ and $\hat{\beta}_1$ in this context?

Note: Hypothesis Testing vs. Prediction

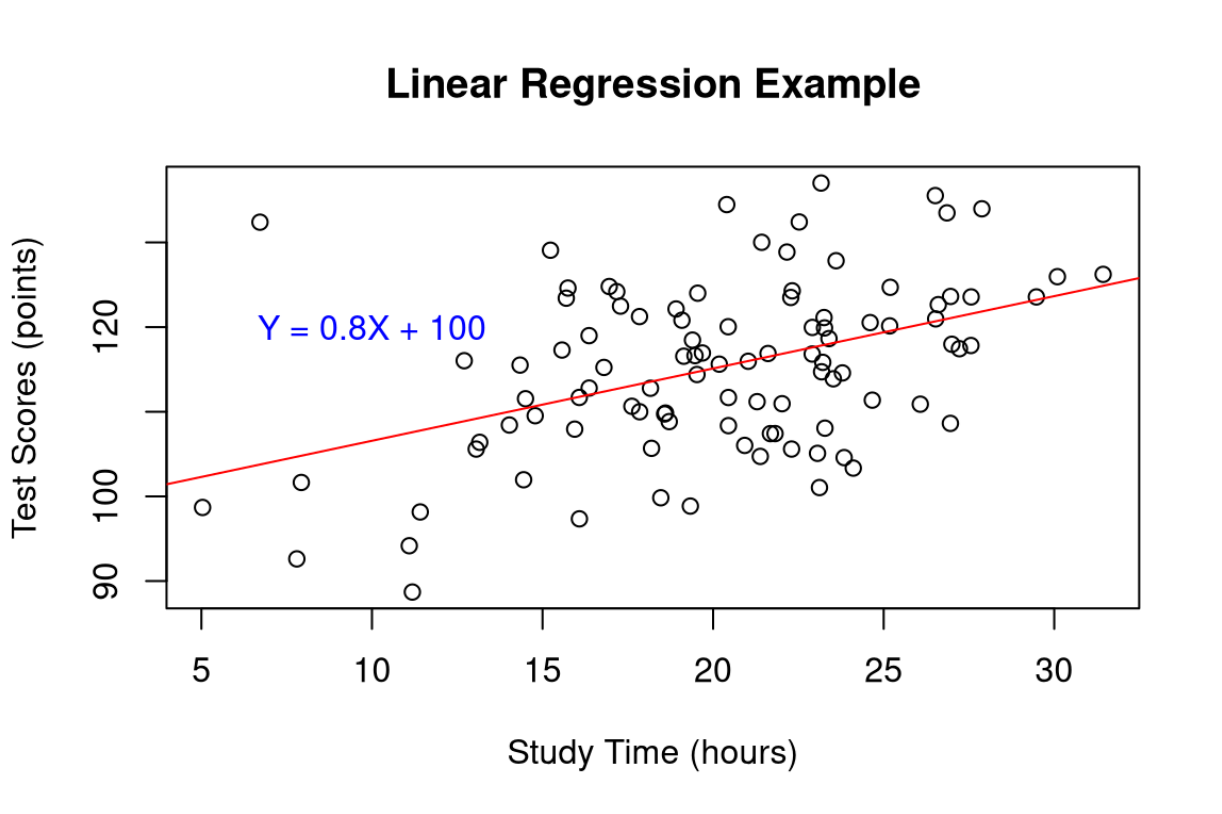
- *Hypothesis Testing:* Focuses on understanding the relationship between variables and testing specific hypotheses about the coefficients (e.g., is β_1 significantly different from zero?).
  - Interpretation of p-value:
    - A low p-value (typically < 0.05) indicates that we can reject the null hypothesis, suggesting that there is a statistically significant relationship between the predictor and outcome variable.
    - A high p-value suggests that we fail to reject the null hypothesis, indicating that there is not enough evidence to support a significant relationship.
- *Prediction:* Focuses on accurately predicting the outcome variable for new data, often prioritizing model performance metrics over statistical significance of coefficients.

Linear Regression Assumptions



Linear Regression Illustration

```r
set.seed(42)
X <- rnorm(100, mean=20, sd=5) # Study time in hours
Y <- 0.8 * X + rnorm(100, mean=100, sd=10)
plot(X, Y, main="Linear Regression Example", xlab="Study Time (hours)", ylab="Test Scores (points)")
text(10, 120, "Y = 0.8X + 100", col="blue")
abline(lm(Y ~ X), col="red")
```



Linear Regression Illustration

```
1  summary(lm(Y ~ X))
```

```
Call:
lm(formula = Y ~ X)

Residuals:
     Min       1Q   Median       3Q      Max
-18.8842  -5.0664   0.1225   5.4106  28.6240

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  98.0300     3.6499  26.858  < 2e-16 ***
X             0.8543     0.1753   4.873 4.24e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 9.083 on 98 degrees of freedom
Multiple R-squared:  0.195, Adjusted R-squared:  0.1868
F-statistic: 23.74 on 1 and 98 DF,  p-value: 4.238e-06
```

Evaluation of Linear Regression

- Given that you want to predict a continuous variable, how do you evaluate how good of a job your model is doing?
  - *R-squared:* Proportion of variance in the outcome variable that is explained by the predictor variable(s).
  - *Mean Absolute Error (MAE):* Average absolute difference between predicted and actual values.
  - *Mean Squared Error (MSE):* Average squared difference between predicted and actual values.

Training and Testing Data

- To evaluate the performance of our model, we typically split our data into a training set and a testing set.
  - The training set is used to fit the model, while the testing set is used to evaluate its performance on unseen data.
- A common split is 80% for training and 20% for testing, but this can vary depending on the size of the dataset and the specific problem you're trying to solve.
  - Your metrics are applied to the testing set to evaluate how well your model generalizes to new data.
  - Failure to properly split data can lead to overfitting
    - Where the model performs well on the training data but poorly on new, unseen data.

Training and Testing Data Illustration

```r
set.seed(42)
X <- rnorm(1000, mean=20, sd=5) # Study time in hours
Y <- 0.8 * X + rnorm(1000, mean=100, sd=10)
training_indices <- sample(1:1000, size=800)
X_train <- X[training_indices] # Training data for predictor variable
Y_train <- Y[training_indices] # Training data for outcome variable
X_test <- X[-training_indices] # Testing data for predictor variable
Y_test <- Y[-training_indices] # Testing data for outcome variable
model <- lm(Y_train ~ X_train) # Fit the linear regression model using the training data
summary(model)
```

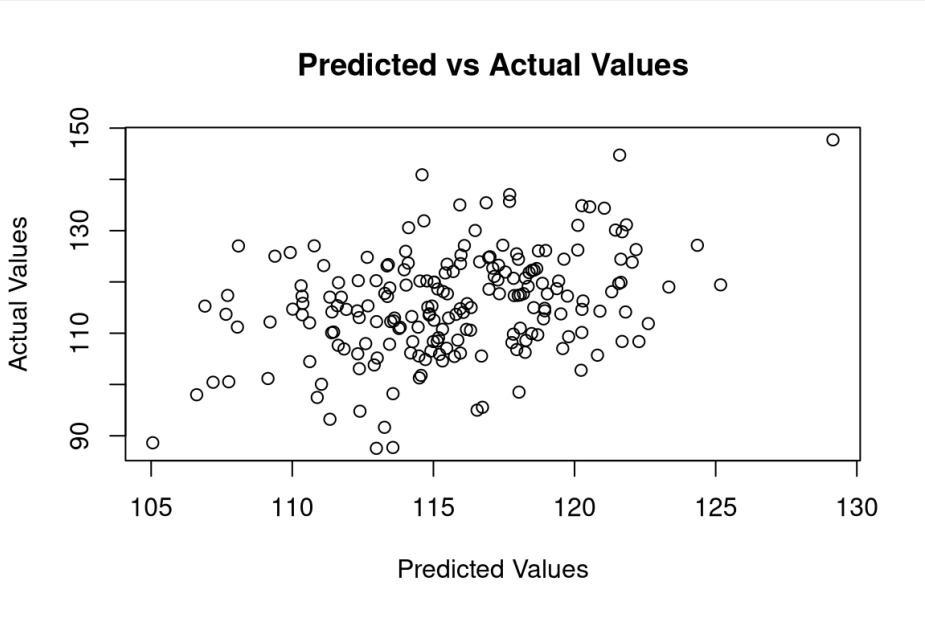
```
Call:
lm(formula = Y_train ~ X_train)

Residuals:
    Min      1Q  Median      3Q     Max
-29.242  -6.579  -0.078   6.716  35.857

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 99.54027    1.42859   69.68   <2e-16 ***
X_train      0.82133    0.06952   11.81   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 9.965 on 798 degrees of freedom
Multiple R-squared:  0.1489,    Adjusted R-squared:  0.1478
F-statistic: 139.6 on 1 and 798 DF,  p-value: < 2.2e-16
```

Evaluating the Model

```r
predictions <- predict(model, newdata=data.frame(X_train=X_test)) # Generate predictions on the testing data
r_squared <- cor(predictions, Y_test)^2 # Calculate the correlation between predictions and actual values in the testi
print(paste("R-squared:", round(r_squared, 2)))
```

```
[1] "R-squared: 0.14"
```

```r
plot(predictions, Y_test, main="Predicted vs Actual Values", xlab="Predicted Values", ylab="Actual Values")
```



- How does this compare with the training data? Is the model performing well on the testing data?

Logistic Regression

- Logistic regression is a statistical method for predicting a binary outcome variable based on one or more predictor variables.
  - Works similarly to linear regression, but instead of fitting a line, it fits an S-shaped curve (sigmoid function) to the data.
- The formula for logistic regression is:

$$P(Y = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X)}}$$

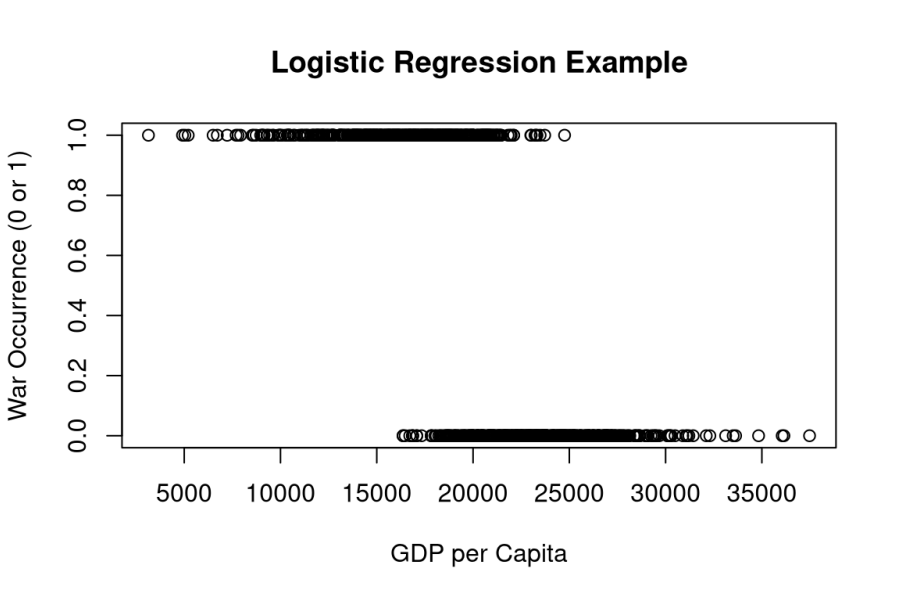
- Where,
  - $P(Y = 1)$ is the probability of the outcome variable being 1 (ex. war occurring, person voting for a candidate, etc.),
  - β_0 is the intercept,
  - β_1 is the coefficient for the predictor variable X.

Evaluating Logistic Regression

- To evaluate the performance of a logistic regression model, we can use metrics such as:
  - *Accuracy:* Proportion of correct predictions (both true positives and true negatives) out of all predictions.
  - *Precision:* Proportion of true positives out of all positive predictions.
  - *Recall (Sensitivity):* Proportion of true positives out of all actual positives.
  - *F1 Score:* Harmonic mean of precision and recall, providing a balance between the two.

Example: Predicting War with Logistic Regression

```r
set.seed(42)
#GDPPC
GDPPC <- rnorm(1000, mean=20000, sd=5000) # GDP per capita
#War occurrence (binary outcome)
War <- 1/(1 + exp(-(10 + -0.0005 * GDPPC + rnorm(1000, mean=0, sd=1)))) # Probability of war occurrence based on GDP pe
War <- ifelse(War > 0.5, 1, 0) # Convert probabilities to binary outcomes (1 for war, 0 for no war)
data <- data.frame(GDPPC, War)
plot(data$GDPPC, data$War, main="Logistic Regression Example", xlab="GDP per Capita", ylab="War Occurrence (0 or 1)")
```



Fit the logistic regression model

```r
train_indices <- sample(1:1000, size=800)
train_data <- data[train_indices, ] # Training dataset
test_data <- data[-train_indices, ] # Testing dataset
model <- glm(War ~ GDPPC, data=train_data, family="binomial") # Fit logistic regre
summary(model)
```

```
Call:
glm(formula = War ~ GDPPC, family = "binomial", data = train_data)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept)  1.704e+01  1.303e+00   13.08   <2e-16 ***
GDPPC       -8.480e-04  6.466e-05  -13.12   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1108.54  on 799  degrees of freedom
```

Example: Evaluating Logistic Regression Model

```r
predictions <- predict(model, newdata=test_data, type="response") # Generate predicted probabilities on the testing dat
predicted_classes <- ifelse(predictions > 0.5, 1, 0) # Convert probabilities to binary predictions
accuracy <- mean(predicted_classes == test_data$War) #
table(predicted_classes, test_data$War) # Confusion matrix
```

```
predicted_classes  0  1
                0 81  9
                1 20 90
```

```r
print(paste("Accuracy:", round(accuracy, 2)))
```

```
[1] "Accuracy: 0.86"
```

Example: Predicting GDP with Light Data

- *Why would we want to predict GDP?*
  - Some countries produce GDP data that is unreliable – North Korea, for example.
  - Data reporting may be interrupted by war, natural disasters, or political instability.
- *What kind of variable is GDP?*
  - A continuous variable
- *What kind of model would we use to predict GDP?*
  - Linear regression, support vector regression, or transformer models.

Exercise time!

- Next, let's apply what we've learned about predicting data and evaluating models to a real-world dataset.